

Student Guide

SOC Analyst Simulation — Web Exploitation: Directory Traversal & Data Exposure

What This Simulation Is Training You To Do

This simulation trains the exact workflow a SOC analyst uses when a web exploitation alert comes in:

- Identify whether traffic is **actually malicious** or just weird-looking noise
- Recognize **directory traversal indicators** (including encoded variants)
- Determine whether exploitation **succeeded** (not just attempted)
- Correlate across **three different log sources**:
 - Web server access logs
 - Web application logs
 - WAF / perimeter logs
- Make a defensible call on **impact** and the **best next action**

The key skill is **evidence-based correlation**: you're proving what happened with logs, not guessing.

The Analyst Workflow

Step 1 — Start in the Access Log: Find “Traversal Intent”

Directory traversal is an attempt to escape the intended web directory to access files like configs, system files, secrets, etc.

Common traversal patterns

- ../
- ..\
- Encoded versions:
 - %2e%2e%2f (../)
 - %2e%2e%5c (..\)
 - ..%2f
- “Double encoding” examples (extra sneaky):
 - %252e%252e%252f (encoded %2e%2e%2f)

Student Guide

What you are trying to prove in this step: “The requester is intentionally trying to break out of the allowed path.”

Use the built-in chip filters

- **Traversal Pattern** chip: quickly isolates rows that look like traversal attempts
- **External Only** chip: helps you ignore internal chatter (in real SOC work, external probes are common)

Step 2 — Separate “Weird URL” From “Actual Attack”

This simulation intentionally includes **false positives** that *look* traversal-ish but are commonly harmless.

Examples of harmless normalization patterns:

- `/images/../../images/logo.png`
- `/assets/css/../../css/main.css`

Those can occur due to:

- Developers using relative paths
- Proxies/CDNs rewriting paths
- Browsers resolving assets weirdly

How you tell the difference:

Ask: *Does it look like it's trying to reach something sensitive (system/config files), or just assets?*

Sensitive targets often look like:

- **/etc/passwd** - Lists local system user accounts, revealing valid usernames and service accounts that attackers can use for credential attacks or lateral movement.
- **/etc/shadow** - Stores hashed user passwords on Linux systems; exposure allows attackers to perform offline password cracking and potentially gain full system access.
- **.env** - Commonly contains application secrets such as database credentials, API keys, and tokens, which can enable further compromise without direct server access.
- **Config.yml** - Often holds application configuration details like service endpoints, credentials, or internal paths that help attackers understand and exploit the system.
- **/proc/self/environ** - Exposes environment variables for the running web process, which frequently include sensitive secrets and credentials loaded at runtime.
- **Win.ini** - A legacy Windows configuration file often targeted to confirm traversal on Windows systems and identify OS type or misconfigurations.

Student Guide

What you are proving in this step: “The target looks sensitive and outside normal web content.”

Step 3 — Determine if it Worked: Attempt vs Success

Lots of traversal attempts fail. You only call **successful exploitation** when you see evidence of an actual sensitive file read.

What “success” often looks like in access logs

Look for a traversal URI plus:

- A “success-looking” status code (commonly 200)
- Bytes returned that make sense for a file response (not tiny error pages)

In the simulation, the **Likely Success** chip helps narrow candidates, but don’t treat it like gospel—verify.

What failure looks like

- 403 (blocked/forbidden)
- 400 (bad request / rejected)
- 404 (not found)
- tiny byte counts that look like standard error pages

Important mindset:

A 200 does **not automatically** mean success — sometimes apps return a “friendly” error page with 200. That’s why we correlate with application logs.

What you are proving in this step: “A request likely returned real content, not an error response.”

Step 4 — Correlate with App Logs: Confirmation of Outcome

After you find your top candidate request in access logs, switch to the **Web Application Log** tab.

You’re looking for results that confirm the app actually did something sensitive.

Strong confirmation indicators

- A result that explicitly suggests sensitive access (for example, something like a sensitive file read)
- A clear app decision indicating the backend processed the request and returned file data
- Session/request timing that aligns closely with the access log request

Student Guide

Defensive indicators (attempt but prevented)

- “Traversal detected”
- “Rejected”
- “Denied”
- “Validation triggered”
- errors that indicate sanitization

What you are proving in this step: “The application backend confirms whether data access occurred.”

Step 5 — Check WAF / Perimeter Logs: Allowed vs Blocked

Now go to the **WAF / Perimeter Log** tab.

You’re answering:

- Did the WAF detect traversal?
- Did it block it?
- If allowed, why might it have passed? (evasion, encoding variation, rule gaps)

A common real-world situation:

- WAF blocks some attempts, but one encoded variant slips through
- That one allowed request matches the successful access/app result

What you are proving in this step: “The perimeter control either failed to stop the successful attempt or never recognized it.”

How to Answer Each Question

1) Attack type identified

Pick **Directory Traversal** when the URI is trying to escape directories.

Don’t pick:

- XSS unless you see script injection patterns
- SQLi unless you see DB query manipulation patterns

2) Request indicating successful exploitation

This should be the **single request** with the strongest combination of:

Student Guide

- Traversal intent (`.. /` or encoded equivalents)
- A status/bytes pattern consistent with real content
- App log corroboration that sensitive content was accessed
- WAF context that shows it was allowed (or at least not blocked effectively)

Tip: You're not picking "the scariest request."
You're picking "the most provably successful request."

3) Resource accessed

This should align with:

- The target file implied by the URI
- The app log evidence

If the URI suggests `.env`, choose `.env`.
If it suggests `/etc/passwd`, choose that.

4) Was authentication bypassed?

High discipline here.

Only answer **Yes** if logs demonstrate:

- a protected resource required auth
- the user had no valid auth context
- and the protected resource was accessed anyway

If logs don't show auth context clearly, **Cannot determine** is often the most defensible.

This is a common SOC trap: people over-claim bypass without proof.

5) Impact classification

Classify impact based on what actually happened:

- Sensitive file read → **Information disclosure**
- No system control changes shown → not privilege escalation
- No outage shown → not denial of service

6) Most appropriate next action

Your best next action should do three things:

1. **Contain risk:** reduce continued exploitation while investigation continues

Student Guide

2. **Preserve evidence:** don't destroy proof you need for scoping and reporting
3. **Remediate root cause:** fix the vulnerable code path, not just a symptom

Blocking a single IP is rarely sufficient because:

- scanners rotate IPs
- other actors may already be probing
- the vulnerability remains exploitable

Password resets are only “best” if evidence suggests credential compromise — which traversal alone does not guarantee.

Technical “Gotchas” This Simulation Is Teaching You

- **Encoded traversal is still traversal:** Attackers rarely use clean `.. /` only. Expect encoding and variations.
- **Not all 200s mean success:** Apps sometimes return “error pages” with 200 responses. Confirm with app logs.
- **One success is enough to be an incident:** Even if 20 attempts failed, one sensitive read is a reportable exposure.
- **Correlation beats instinct:** If you can't cite which log rows prove your conclusion, you're guessing.

Analyst Mindset Reminder

You're training two skills that separate good analysts from guessers:

- **Precision:** “What happened” is not the same as “what could have happened.”
- **Defensibility:** Your conclusion should be provable using the logs in front of you.